

Guided Project: PDF-Based Knowledge Base RAG System

Duration: 60 Min.

Scenario

Build a complete Retrieval-Augmented Generation (RAG) pipeline using LangChain, OpenAI (`text-embedding-3-small` and `gpt-4o-mini`), and ChromaDB. The system must ingest an employee handbook PDF, create vector embeddings, retrieve relevant context, and provide context-grounded answers while refusing to hallucinate when asked questions outside the document.

Target File Location & Creation

- **Solution Script:** `student_workspace/solution.py` (Complete Tasks 1 to 4)
- **Application File:** `student_workspace/rag_app.py` (Complete standalone interactive application - Task 5)
- **Source Document:**
`student_workspace/04_DATA/sample_knowledge_base.pdf`

Required Pipeline

PDF → PyPDFLoader → RecursiveCharacterTextSplitter (1000/200) → `text-embedding-3-small` → ChromaDB → Retriever (Top-3) → GPT-4o-mini → Grounded Answer + Source Page

Instructions to Perform the Task

1. When your workspace loads in **VS Code**, use the **Explorer** panel on the left to locate files in `student_workspace/`.

2. Open `student_workspace/solution.py` and implement the guided functions for Tasks 1 to 4:
 - **Task 1:** Load PDF using `PyPDFLoader`
 - **Task 2:** Chunk documents with `RecursiveCharacterTextSplitter` (`chunk_size=1000`, `chunk_overlap=200`)
 - **Task 3:** Generate embeddings with `OpenAIEmbeddings(model="text-embedding-3-small")` and persist in ChromaDB
 - **Task 4:** Construct grounded RAG pipeline with `gpt-4o-mini` and exact refusal fallback
3. Assemble the complete standalone interactive CLI application in `student_workspace/rag_app.py` (Task 5) with query loop and `exit` condition.
4. Save your files (`Ctrl + S` or `Cmd + S`).
5. Open the built-in terminal (**Terminal > New Terminal**) and verify your implementation locally by running:

```
python run.py
```

6. Once all test cases pass, return to the platform dashboard and click the **"Run Test"** / **"Verify"** button to submit.

Validation & Scoring Rubric

Test Case	Requirement	Marks
TC1	Task 1: PDF Loading & Processing (<code>PyPDFLoader</code> , extracts 15 pages, preserves metadata)	5 Marks
TC2	Task 2: Text Chunking (<code>RecursiveCharacterTextSplitter</code> , <code>chunk_size=1000</code> , <code>chunk_overlap=200</code>)	20 Marks
TC3	Task 3: Embeddings & ChromaDB (<code>text-embedding-3-small</code> , Chroma persistence, similarity search)	25 Marks
TC4	Task 4: Grounded RAG Pipeline (Top-3 retrieval, <code>gpt-4o-mini</code> , grounded prompt, exact refusal fallback)	25 Marks
TC5	Task 5: Final Interactive App (<code>rag_app.py</code>) (Complete CLI loop, answers queries, handles exit)	25 Marks

